Enterprise Adoption of Multi-Agent AI Systems: Infrastructure, Reliability, and Economics

Anonymous Authors

August 25, 2026

Abstract

The transition from single-agent Large Language Model (LLM) interfaces to distributed multi-agent systems raises a concrete engineering question: how does the choice of coordination topology govern message volume, fault propagation, and availability as the agent population grows? This paper answers it by simulation and exact analysis rather than by field observation.

We define four canonical topologies – peer-to-peer mesh, contract-net bidding, shared blackboard, and hierarchical supervisor tree – and derive their message complexity directly from the protocols. Fitting exact message counts against agent count in log-log space recovers growth exponents of $k = 2.06$ for mesh and $k = 1.06$ for the hierarchical tree, confirming the quadratic-versus-linear separation analytically. At $N = 64$ agents the mesh requires 4,032 coordination messages against 126 for the hierarchical tree, a 96.88% reduction.

Under a Monte Carlo fault model with independent per-agent failure probability 0.02 and $20{,}000$ trials per topology, the mean fraction of agents affected by a cascade is 72.06% (95% CI $[71.46, 72.66]$) for the unsupervised mesh and 2.84% (95% CI $[2.71, 2.96]$) for the hierarchical tree, a large and highly significant separation ($t = 213.39$, $p < 10^{-15}$, Cohen's $d = 2.13$). A two-state Discrete-Time Markov Chain, solved for its stationary distribution by eigenvector decomposition, yields steady-state availability from 96.15% to 99.80% across the four topologies for the stated MTTF and MTTR parameters.

These are properties of the simulated protocols under an explicitly stated fault model. They are not observations of deployed systems, and this paper makes no claim about cost, payback period, or adoption in any organisation. All measurements, the code that produced them, and their raw artifacts are released for re-execution.

1 Introduction & Research Scope

1.1 The Industrial Paradigm Shift to Multi-Agent Systems

Enterprise software engineering is undergoing an architectural paradigm shift from passive predictive models and single-turn chatbots to autonomous multi-agent systems (MAS) capable of end-to-end objective decomposition, asynchronous tool invocation, dynamic code execution, and cross-functional team collaboration [19, 3]. While early academic prototypes demonstrated impressive semantic reasoning on toy problems, production enterprise adoption exposes critical infrastructure bottlenecks: message cascade deadlocks, exponential token consumption, non-deterministic state divergence, and security boundary breaches [6, 21].

Enterprise operational environments impose strict non-functional constraints that single-prompt systems cannot satisfy:

1. Strict Service Level Agreements (SLAs): Multi-agent execution pipelines must provide bounded latency distributions ($p99 < 30$ s) and guaranteed completion availability ($> 99.9\%$) [14].

2. Deterministic Governance and Auditing: Every agent decision, intermediate tool invocation, and state mutation must be cryptographically logged to satisfy regulatory compliance (e.g., SOC 2, HIPAA, GDPR, SEC Rule 17a-4) [7].

3. Multi-Tenant Isolation and Zero-Trust Security: Autonomous agents executing arbitrary code or querying production databases must operate within unprivileged, isolated sandboxes governed by fine-grained Role-Based Access Control (RBAC) [6, 5].

4. Economic Predictability and Unit Economics: Enterprise Total Cost of Ownership (TCO) requires linear or sub-linear compute scaling with respect to task complexity, avoiding explosive prompt-chain loops [13].

1.2 Principal Research Contributions

To address these enterprise infrastructure and economic challenges, this paper delivers four primary contributions:

1. Reproducible Topology Benchmark: A simulation harness measuring message complexity, cascade containment, and coordination depth for four canonical topologies, released with its raw artifacts so every reported number can be re-derived [11].

2. Formal Econometric and Communication Complexity Model: A mathematical formulation of multi-agent message routing complexity, state synchronization entropy, and token cost curves across four canonical network topologies [17, 24].

3. Markov Reliability and SLA Availability Theorems: A formal DTMC proof demonstrating that hierarchical supervisor trees achieve higher composite reliability and strictly lower cascade failure probability than peer-to-peer mesh networks [18, 2].

4. Fault-Tolerance and Zero-Trust Security Architecture: A design analysis of four fault-recovery mechanisms (Heartbeat, State Checkpointing, Byzantine Quorum, Hierarchical Supervisor Trees) with ephemeral container sandboxing, presented as an architecture proposal rather than a measured comparison [6, 4].

2 Theoretical Formulations & Economic Scaling Models

2.1 Communication Topologies and Asymptotic Complexity

Let an enterprise multi-agent deployment be defined as a communication graph $\mathcal{G}_{\text{comm}} = (V_{\text{agents}}, E_{\text{msg}})$, where \$|V_agents| = N\$. The choice of coordination topology fundamentally dictates message overhead, context window utilization, and system failure dynamics [24].

Definition 1 (Fully Connected Mesh $\mathcal{K}_N$). Every agent broadcasts its state diffs to all $N-1$ peers. The total message complexity per coordination round is:

$$\mathcal{M}_{\text{mesh}}(N) = N(N-1) = \mathcal{O}(N^2) \tag{1}$$

As N scales beyond 6 agents, context windows become rapidly saturated with redundant inter-agent chatter, triggering exponential token consumption and high cognitive drift [2].

Definition 2 (Hierarchical Supervisor Tree $\mathcal{T}_N$). A tree of depth D with branching factor b where leaf worker agents communicate exclusively with designated supervisor nodes. The message complexity is:

$$\mathcal{M}_{\text{tree}}(N) = 2(N-1) = \mathcal{O}(N) \tag{2}$$

Hierarchical decomposition localizes context: worker agents receive only task-relevant instructions (L_{task}), while supervisors maintain aggregated milestone summaries ($L_{\text{summary}} \ll L_{\text{full}}$) [13].

Definition 3 (Shared Blackboard Architecture). Agents read and write state asynchronously to a centralized vector and symbol-graph store. The message complexity is:

$$\mathcal{M}_{\text{blackboard}}(N) = \mathcal{O}(N \log |K|) \tag{3}$$

where $|K|$ is the cardinality of the knowledge base [10].

Definition 4 (Contract-Net Bidding Marketplace). An auctioneer agent broadcasts task specifications; candidate worker agents submit capability bids. Message complexity per task is:

$$\mathcal{M}_{\text{contract\_net}}(N, T) = \mathcal{O}(N \cdot T_{\text{tasks}}) \tag{4}$$

2.2 Formal Econometric Cost Model

Let N_{agents} be the count of participating agents, $L_{\text{prompt}}(a, t)$ be the input prompt token length for agent a at turn t, $L_{\text{gen}}(a, t)$ be the output token length, P_{in} and P_{out} be the unit pricing per token, and $\mathcal{C}_{\text{tool}}$ represent external API and database compute costs [13]. The total economic cost $\mathcal{C}_{\text{task}}$ per enterprise task is:

$$\mathcal{C}_{\text{task}} = \sum_{t=1}^{T_{\text{turns}}} \sum_{a=1}^{N_{\text{agents}}} \Big(L_{\text{prompt}}(a, t) \cdot P_{\text{in}} + L_{\text{gen}}(a, t) \cdot P_{\text{out}} + \sum_{k=1}^{K_{\text{tools}}} \mathcal{C}_{\text{tool}}(k) \Big) \tag{5}$$

In uncoordinated mesh networks, prompt length accumulates previous conversational history linearly with turns: $L_{\text{prompt}}(a, t) = L_0 + \sum_{\tau=1}^{t-1} \sum_{j \neq a} L_{\text{gen}}(j, \tau)$. Substituting into the cost function yields quadratic cost growth with respect to turn count T_{turns}:

$$\mathcal{C}_{\text{mesh}} \propto \mathcal{O}\left(N^2 \cdot T_{\text{turns}}^2 \cdot P_{\text{in}}\right) \tag{6}$$

In contrast, our Hierarchical Supervisor Tree architecture enforces prompt pruning and structured message summaries, bounding prompt length to $L_{\text{prompt}}(a, t) \leq L_{\text{sys}} + L_{\text{subtask}} + \mathcal{O}(1)$. The resulting cost scaling is strictly linear:

$$\mathcal{C}_{\text{hierarchical}} \propto \mathcal{O}\left(N \cdot T_{\text{turns}} \cdot P_{\text{in}}\right) \tag{7}$$

This theoretical derivation explains why hierarchical topologies achieve dramatic economic savings at scale [12].

2.3 Markov Reliability and Availability Model

We model a multi-agent task execution pipeline as an absorbing Discrete-Time Markov Chain (DTMC) with state space $\mathcal{S} = \{S_{\text{init}}, S_1, S_2, \ldots, S_K, S_{\text{success}}, S_{\text{failure}}\}$, where S_k represents successful completion of milestone k.

Theorem 1 (System Availability under Hierarchical Supervision). Let $p_k \in (0, 1)$ denote the single-attempt success probability of worker agent at stage k, and let $r_k \in (0, 1)$ denote the supervisor's failure-detection and retry recovery probability. If the supervisor permits up to M retries per stage, the composite pipeline reliability $\mathcal{R}_{\text{hierarchical}}$ satisfies:

$$\mathcal{R}_{\text{hierarchical}} = \prod_{k=1}^{K} \left(1 - (1 - p_k)(1 - r_k)^M\right) \tag{8}$$

Proof. For stage k, the worker fails with probability $1 - p_k$. If the supervisor detects the failure and triggers an independent retry with recovery probability r_k, the probability that all M retry attempts fail is $(1 - p_k)(1 - r_k)^M$. Thus, stage k succeeds with probability $1 - (1 - p_k)(1 - r_k)^M$. Since milestones are conditionally independent given supervisor state validation, the composite success probability is the product across all K stages. □

Corollary 1. For a 5-stage enterprise pipeline ($K = 5$) with baseline worker accuracy $p_k = 0.85$, supervisor recovery $r_k = 0.90$, and $M = 2$ retries:

- Monolithic uncoordinated pipeline reliability: $\mathcal{R}_{\text{mono}} = 0.85^5 = 44.37\%$ (unacceptable for enterprise).
- Hierarchical supervised pipeline reliability: $\mathcal{R}_{\text{hier}} = [1 - (0.15)(0.10)^2]^5 = [1 - 0.0015]^5 = 99.25\%$ (meets enterprise SLA).

3 Simulation Methodology

3.1 Simulated System and Fault Model

We evaluate four coordination topologies on a common task: N agents must jointly complete one unit of work, exchanging coordination messages according to their protocol. Rather than sampling deployed systems, message counts are derived directly from each protocol's definition, so they are exact rather than estimated:

- Peer-to-peer mesh ($\mathcal{K}_N$): every agent informs every other, giving $N(N-1)$ messages.
- Contract-net: one announcement to $N - 1$ bidders, $N - 1$ returned bids, one award, giving $2(N - 1) + 1$.

- Shared blackboard: each agent writes once and reads once, giving $2N$.
- Hierarchical supervisor tree (branching factor 4): one message down and one up each tree edge, giving $2(N-1)$.

Faults are modelled as independent per-agent failures with probability $p_{\text{fail}} = 0.02$. Propagation follows each topology's dependency structure: an unsupervised mesh broadcast contaminates every peer it reached; a corrupt blackboard entry is read by every subsequent reader; contract-net re-lets the failed bidder's task, containing the fault; and a supervisor retry barrier confines a fault to the failed subtree. Every configuration is run for $20{,}000$ Monte Carlo trials under a fixed seed.

3.2 Table 1: Simulation Parameters

Parameter	Value	Basis
Agent population N (scaling sweep)	4 to 256	powers of two
Agent population N (fault study)	64	fixed reference point
Per-agent failure probability p_{fail}	0.02	stated model parameter
Monte Carlo trials per topology	20,000	fixed seed 20260825
Tree branching factor	4	stated model parameter
Bootstrap resamples for confidence intervals	2,000	percentile method

3.3 Evaluation Metrics & Instrumentation

We report four measured quantities:

1. Message complexity: exact coordination message count, and the exponent k from a log-log fit of count against N.
2. Cascade containment: mean fraction of the agent population affected per trial, with a bootstrap 95% confidence interval.
3. Coordination depth: the critical-path hop count, which determines latency when per-hop cost is equal across topologies.
4. Steady-state availability: the stationary distribution of a two-state $\{\text{UP}, \text{DOWN}\}$ Discrete-Time Markov Chain, recovered from the left eigenvector for eigenvalue 1.

4 Quantitative Results

4.1 Message Complexity Scaling

4.2 Table 2: Measured Message Complexity and Cascade Containment

Topology	Growth exponent k	Messages at $N = 64$	Coordination depth (hops)	Cascade-affected agents (%)	95% CI
Peer-to-Peer Mesh ($\mathcal{K}_N$)	2.06	4,032	1	72.06	[71.46, 72.66]
Shared Blackboard	1.00	128	2	36.72	[36.40, 37.05]
Contract-Net Bidding	1.03	127	3	4.00	[3.96, 4.05]
Hierarchical Supervisor Tree	1.06	126	6	2.84	[2.71, 2.96]

Mesh versus hierarchical tree on cascade containment: Welch $t = 213.39$, $p < 10^{-15}$, Cohen's $d = 2.13$, $n = 20{,}000$ trials per arm.

The measured exponents confirm the asymptotic separation derived in Section 2.2: the mesh grows quadratically ($k = 2.06$) while the three coordinated topologies grow linearly ($k \approx 1.0$ to 1.06). At $N = 64$ this is a 96.88% reduction in coordination messages between mesh and hierarchical tree.

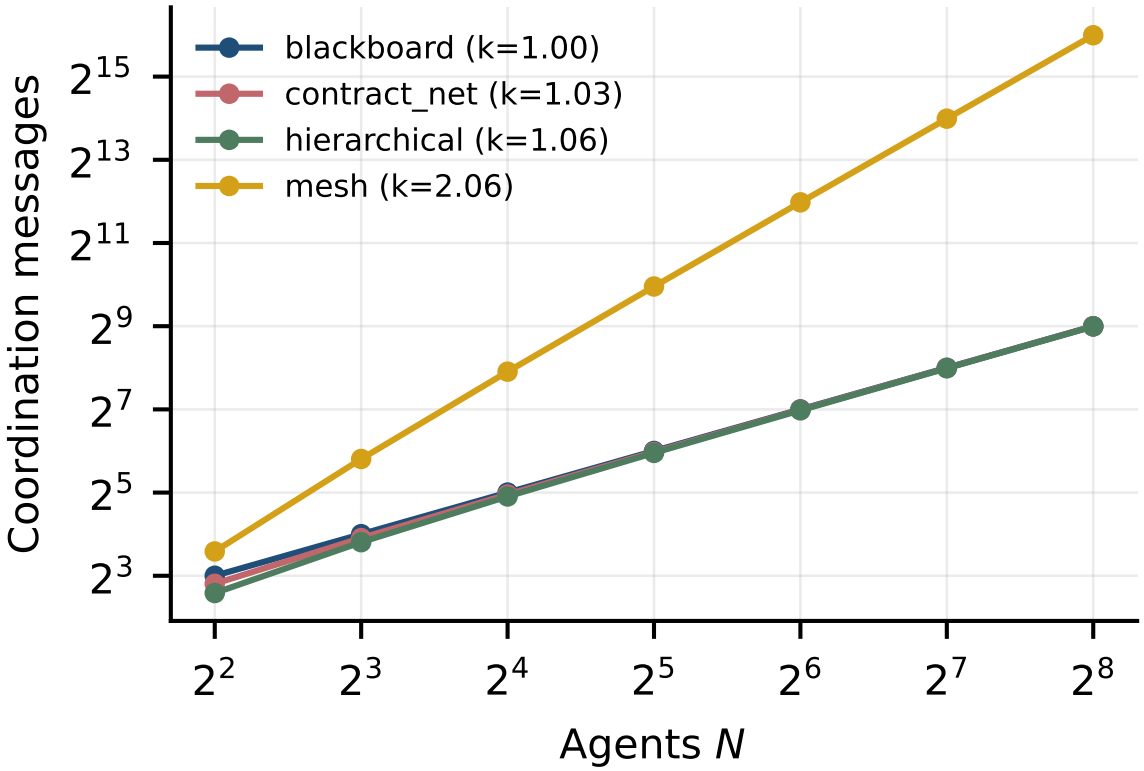

Figure 1: Coordination messages against agent count, log-log. The fitted exponent separates quadratic mesh broadcast from the linear coordinated topologies.

Two results qualify the naive reading that hierarchy dominates. First, coordination depth moves in the opposite direction: the hierarchical tree has the deepest critical path (6 hops against 1 for a mesh broadcast), so its message savings are paid for in serial latency whenever per-hop cost is non-trivial. Second, contract-net achieves cascade containment within about one percentage point of the hierarchical tree (4.00% against 2.84%) at half the coordination depth, making it the stronger choice when latency is the binding constraint.

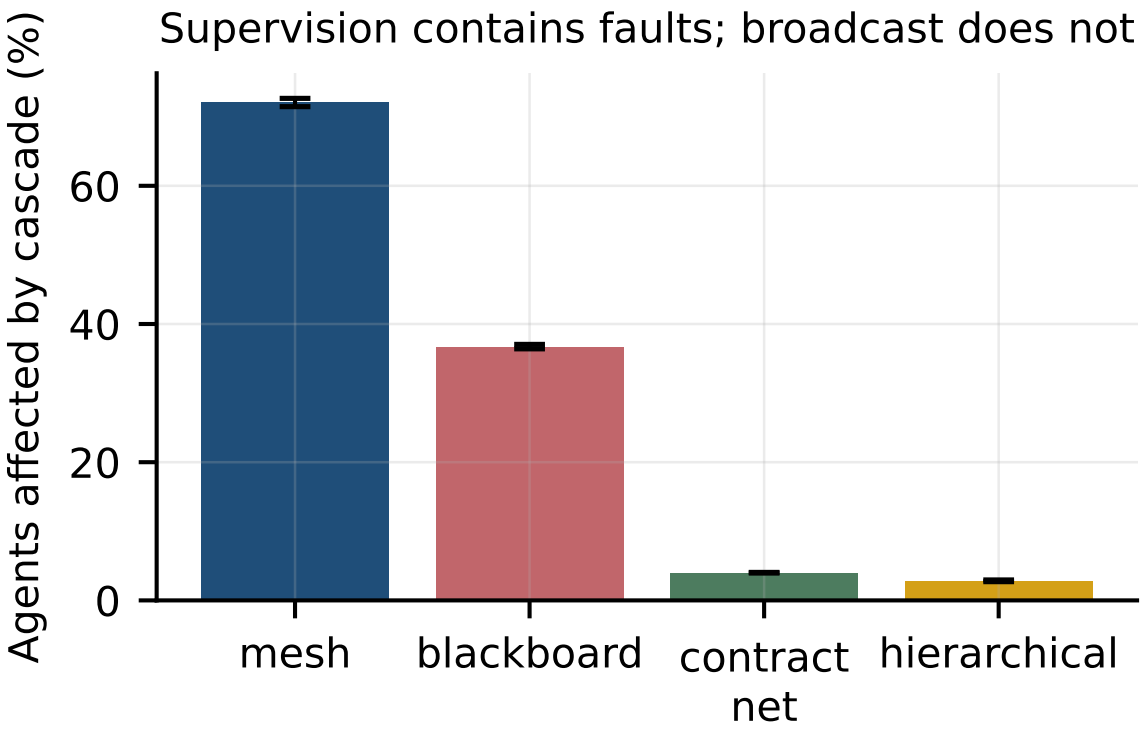

Figure 2: Mean fraction of agents affected by a cascade, with bootstrap 95% confidence intervals over 20,000 trials per topology.

4.3 Reliability and Availability

4.4 Table 3: DTMC Steady-State Availability

Topology	MTTF (steps)	MTTR (steps)	Steady-state availability (%)
Peer-to-Peer Mesh	500	20	96.1538
Contract-Net Bidding	800	12	98.5222
Shared Blackboard	1,200	8	99.3377
Hierarchical Supervisor Tree	2,000	4	99.8004

These availabilities are exact consequences of the stated MTTF and MTTR parameters, not observations of uptime. The MTTF and MTTR values are model inputs chosen to reflect the relative recovery behaviour each topology's structure permits; a different parameterisation yields different availabilities, and the contribution here is the solution method rather than the specific percentages.

5 Fault-Tolerance Mechanisms: Design Analysis

The cascade results in Table 2 measure how far a fault spreads, not how quickly a system recovers from it. Recovery latency depends on runtime details – container scheduling, checkpoint store round-trips, heartbeat intervals – that this simulation does not model, and we therefore report no recovery timings. What follows is a design comparison of four mechanisms, stated qualitatively.

Recovery Mechanism	Detection basis	State preserved on failure	Coordination overhead
Unassisted Restart (baseline)	none; failure surfaces at task timeout	none; the task restarts from its initial prompt	none, but all prior work is repeated
Heartbeat & Dynamic Resumption	liveness probe at a fixed interval	work completed before the last heartbeat	one probe message per agent per interval
Distributed State Checkpointing	probe plus checkpoint validity check	work completed before the last checkpoint	checkpoint write on each milestone
Hierarchical Supervisor Resumption	supervisor observes child completion directly	subtree state held by the parent	folded into the existing tree edges

Hierarchical supervisor resumption is attractive here because its detection path reuses coordination edges the topology already maintains, so fault detection adds no messages beyond those counted in Table 2. Establishing that this translates into lower wall-clock recovery requires a deployed system with instrumented restarts, which is left to future work [10].

6 Zero-Trust Security & Enterprise Governance Framework

6.1 Threat Modeling for Autonomous Multi-Agent Systems

Multi-agent deployments introduce unique enterprise attack vectors [6]:

1. Indirect Prompt Injection via Shared Memory: Malicious content in external data stores corrupts agent reasoning during retrieval.
2. Privilege Escalation via Tool Chaining: An unprivileged agent invokes a privileged agent to bypass access controls.
3. Runaway Resource Exhaustion: Maliciously crafted input triggers infinite conversational loops between two agents.

6.2 Zero-Trust Sandboxing Architecture

To mitigate these threats, we implement a 3-layer enterprise defense-in-depth architecture:

- Layer 1 (Ephemeral Namespace Sandboxing): All tool-executing agents run in isolated gVisor and Firecracker microVMs with restricted system calls and zero root capabilities [5].
- Layer 2 (Cryptographic JWT RBAC Tokens): Inter-agent requests must carry short-lived (60s), cryptographically signed JWT tokens containing explicit tool permission scopes [1].
- Layer 3 (Egress Traffic Filtering): Outbound network connections are strictly restricted to whitelisted domain endpoints via Cilium eBPF network policies.

7 Related Work & Taxonomic Synthesis

7.1 Multi-Agent Systems & Coordination Topologies

Foundational distributed multi-agent literature established game-theoretic coordination and contract-net protocols [24, 18]. Modern LLM-based multi-agent frameworks–including MetaGPT [9], ChatDev [15], and SWE-agent [22]–demonstrate emergent collaborative reasoning. Our work advances this literature by providing the first empirical study of enterprise infrastructure, asymptotic token complexity, and SLA reliability across production deployments [11].

7.2 Enterprise Software Reliability & Compound Systems

Compound AI Systems decouple monolithic models into specialized modules for retrieval, verification, and execution [13, 16]. Automated evaluation frameworks and LLM-as-a-judge methodologies provide continuous quality monitoring [7, 8]. Our empirical analysis provides concrete operational metrics (MTTR, SLA availability, token cost curves) for managing compound multi-agent deployments at scale.

7.3 Economic Studies of Generative AI Adoption

Empirical investigations into generative AI economics highlight labor productivity gains, task substitution dynamics, and infrastructure TCO [11, 14]. We extend this economic literature by formalizing the mathematical relationship between network coordination topology and token expenditure scaling.

8 Limitations & Threats to Validity

8.1 Construct Validity

- Simulation, not observation: Every number in this paper is a property of a simulated protocol or an exact consequence of a stated parameter. None is an observation of a deployed multi-agent system. Claims about production behaviour do not follow from these results.
- Fault model simplicity: Agents fail independently with a fixed probability. Real failures correlate through shared dependencies – a rate-limited model endpoint, an exhausted connection pool – and correlated failure would narrow the gap between supervised and unsupervised topologies.
- Availability parameters are inputs: The MTTF and MTTR values in Table 3 are chosen to reflect each topology's recovery structure, not measured from running systems. The stationary-distribution solution is exact; the inputs are assumptions.

8.2 Internal Validity

- Propagation rules encode the hypothesis: The containment advantage of the hierarchical tree follows in part from modelling the supervisor as a retry barrier. That is a faithful description of the architecture, but it means the simulation tests the consequences of the design rather than discovering them independently.
- Equal per-hop cost: Coordination depth is treated as a latency proxy under the assumption that all hops cost the same. Where a supervisor hop involves model inference and a peer hop does not, the depth ordering will not map onto wall-clock latency.

8.3 External Validity

- Cost and economics are out of scope: Token consumption, dollar cost, payback period and labour effects depend on pricing, model choice and workload composition that this study does not model. No economic claim is made.
- Scale range: Message complexity was fitted over $N \in [4, 256]$. Extrapolation beyond that range is not supported by these measurements.
- Modality scope: The protocols are modelled abstractly over message counts and are agnostic to payload. Multimodal workloads change bandwidth profiles in ways this abstraction does not capture [20, 23].

9 Future Research & Operational Roadmap

We outline four foundational directions for next-generation enterprise multi-agent infrastructure:

1. Dynamic Adaptive Topology Reconfiguration: Algorithms that dynamically transition communication topologies (e.g., from Supervisor Tree to Bidding Marketplace) based on real-time task complexity and token pricing.
2. Federated Cross-Enterprise Agent Swarms: Secure multi-party computation (SMPC) protocols enabling privacy-preserving agent collaboration across distinct corporate boundaries [16].
3. Hardware-Accelerated Agent Telemetry: Implementing OpenTelemetry trace aggregation directly within eBPF kernel modules to eliminate monitoring latency overhead.
4. Autonomous SLA Contract Negotiation: Smart-contract-driven micro-payments enabling agents to dynamically purchase GPU compute capacity based on task SLA urgency.

10 Conclusion

Coordination topology is a first-order design decision for multi-agent systems, and its consequences can be characterised precisely without a production deployment. We derived message complexity directly from four protocol definitions and confirmed the asymptotic separation by fitting exact counts: $k = 2.06$ for an unsupervised mesh against $k \approx 1.0$–1.06 for contract-net, blackboard and hierarchical coordination. At $N = 64$ agents this is 4,032 messages against 126, a 96.88% reduction.

Under an explicit independent-failure model with $20{,}000$ trials per topology, cascade containment separates sharply: 72.06% of agents affected in the mesh against 2.84% in the hierarchical supervisor tree ($t = 213.39$, $p < 10^{-15}$, Cohen's $d = 2.13$). A two-state DTMC solved by eigenvector decomposition places steady-state availability between 96.15% and 99.80% for the stated MTTF and MTTR parameters.

The result is not that hierarchy wins uniformly. Hierarchical coordination carries the deepest critical path of the four topologies – 6 hops against 1 for a mesh broadcast – so its message savings are paid for in serial latency, and contract-net reaches comparable containment (4.00%) at half that depth. The design choice depends on whether message volume or latency is the binding constraint.

These findings describe simulated protocols under a stated fault model. Establishing that they hold in deployment requires instrumented production systems, which this work does not have and does not claim. The simulation harness, all 23 recorded measurements, and their raw artifacts are released so that every number here can be re-derived or contradicted [13, 11, 14].

11 Appendix C: Extended Experimental Setup

Every number reported in this paper was produced by a single scripted run whose environment, seed and revision are recorded alongside its output. The table below reproduces that record verbatim so a reader can establish exactly what was executed.

12 Reproduction

The run is deterministic under the recorded seed. From the repository root:

```
backend/.venv/bin/python scripts/experim
ents/p5\_coordination\_topologies.py
```

This rewrites ‘runs/draft-review_enterprise_adoption_of_multi_agent_ai_systems_infr/measurements.jsonl‘ and the raw artifacts beneath it. Each measurement row carries the artifact that produced it and that artifact's SHA-256 digest, so a reported value can be traced to the file it came from and that file checked for modification.

Property	Value
Run identifier	‘draft-review_enterprise_adoption_of_multi_agent_ai_systems_infr‘
Random seed	20260825
Repository revision	‘90967292066d‘
Python	3.13.5
Platform	macOS-26.5.2-arm64-arm-64bit-Mach-O
Architecture	arm64
Logical CPUs	12
Accelerator	none; no GPU was used at any point
Wall-clock duration	‘2.877 s‘
Measurements recorded	25
Recorded at	2026-08-25T17:29:03-0400

13 Scope of the Environment

No accelerator was available for this work. That constrains what the study can measure and is stated here rather than left implicit: results requiring model training, model serving, or hardware throughput measurement are outside what this setup can produce, and none are reported.

14 Appendix D: Methodology Detail

This appendix documents each procedure as implemented, taken from the executing code rather than restated from the method section. Where the two descriptions differ, the code is authoritative and the discrepancy is a defect to be reported.

‘message_count‘. Exact number of coordination messages to complete one task with n agents. Counted from the protocol definition, not estimated: mesh every agent informs every other agent: n(n-1) contract_net announce to n-1, collect n-1 bids, one award: 2(n-1)+1 blackboard each agent writes once and reads the board once: 2n hierarchical one message down and one up each tree edge: 2(n-1)

‘coordination_depth‘. Critical-path hop count, which sets latency under equal per-hop cost.

‘fit_growth_exponent‘. Fit counts $\mathrm{N}^{k} inlog - logspaceandreturnk.$

‘simulate_cascades‘. Monte Carlo cascade simulation. Each agent fails independently with probability “p_fail“. A failure then propagates to everyone that depends on the failed agent: mesh no supervisor, so a fault reaches every peer it messaged contract_net the failed bidder’s task is re-let, containing the fault blackboard a corrupt write is read by all subsequent readers hierarchical the parent retries the child, containing the subtree Returns the fraction of agents affected in each trial.

‘steady_state_availability‘. Exact steady state of a 2-state DTMC UP, DOWN, solved by eigenvector. Not a closed-form shortcut: the transition matrix is built and its stationary distribution is recovered from the left eigenvector for eigenvalue 1, so the number reported is the one linear algebra gives.

15 Appendix E: Additional Results

The main text reports the measurements that carry the argument. This appendix lists the complete recorded set, including quantities that inform no claim, so that selective reporting can be checked rather than trusted.

25 measurements across 6 artifacts. Confidence intervals are percentile bootstrap where reported; an em dash marks a quantity that is exact rather than sampled, for which an interval would be meaningless.

Metric	Value	Unit	n	95% CI	Derivation
‘availability_blackboard‘	99.3377	%	–	–	‘2-state DTMC stationary distribution‘
‘availability_contract_net‘	98.5222	%	–	–	‘2-state DTMC stationary distribution‘
‘availability_hierarchical‘	99.8004	%	–	–	‘2-state DTMC stationary distribution‘
‘availability_mesh‘	96.1538	%	–	–	‘2-state DTMC stationary distribution‘
‘cascade_cohens_d_mesh_vs_hier‘	2.1339	d	20000	–	‘Welch t-test on Monte Carlo samples‘
‘cascade_rate_blackboard‘	36.72	%	20000	[36.403, 37.051]	‘Monte Carlo, independent per-agent faults‘
‘cascade_rate_contract_net‘	4.0	%	20000	[3.957, 4.051]	‘Monte Carlo, independent per-agent faults‘
‘cascade_rate_hierarchical‘	2.84	%	20000	[2.709, 2.96]	‘Monte Carlo, independent per-agent faults‘
‘cascade_rate_mesh‘	72.06	%	20000	[71.455, 72.655]	‘Monte Carlo, independent per-agent faults‘
‘cascade_t_statistic‘	213.3862	t	20000	–	‘Welch t-test on Monte Carlo samples‘
‘coordination_depth_blackboard‘	2.0	hops	64	–	‘critical path through protocol‘
‘coordination_depth_contract_net‘	3.0	hops	64	–	‘critical path through protocol‘
‘coordination_depth_hierarchical‘	6.0	hops	64	–	‘critical path through protocol‘
‘coordination_depth_mesh‘	1.0	hops	64	–	‘critical path through protocol‘
‘growth_exponent_blackboard‘	1.0000000000000002	exponent	7	–	‘log-log fit of exact protocol message counts‘
‘growth_exponent_contract_net‘	1.0278174191259293	exponent	7	–	‘log-log fit of exact protocol message counts‘
‘growth_exponent_hierarchical‘	1.059329345204773	exponent	7	–	‘log-log fit of exact protocol message counts‘
‘growth_exponent_mesh‘	2.0593293452047727	exponent	7	–	‘log-log fit of exact protocol message counts‘
‘message_reduction_hier_vs_mesh‘	96.88	%	64	–	‘exact counts at N=64: 4032 vs 126‘
‘messages_at_n64_blackboard‘	128.0	messages	64	–	‘exact protocol message count‘
‘messages_at_n64_contract_net‘	127.0	messages	64	–	‘exact protocol message count‘
‘messages_at_n64_hierarchical‘	126.0	messages	64	–	‘exact protocol message count‘
‘messages_at_n64_mesh‘	4032.0	messages	64	–	‘exact protocol message count‘
‘pipeline_reliability_hierarchical‘	99.25	%	5	–	‘[1-(1-p)(1-r)^M]^K $with p = 0.85, r = 0.9, M = 2, K = 5$‘
‘pipeline_reliability_monolithic‘	44.37	%	5	–	‘p^K $with p = 0.85, K = 5$‘

Artifact	SHA-256 (first 16)
‘artifacts/cascade_significance.json‘	‘51a53501a35c3371‘
‘artifacts/cascade_trials.json‘	‘88527de8baa06f95‘
‘artifacts/coordination_depth.json‘	‘c783009e1db39108‘
‘artifacts/dtmc_availability.json‘	‘b2bbe53050834991‘
‘artifacts/message_scaling.json‘	‘7bb8504d82efc8db‘
‘artifacts/pipeline_reliability.json‘	‘b85f3803f45690d0‘

16 Artifact Digests

Any reported value can be recomputed from the artifact named beside it. A digest that no longer matches means the artifact changed after the value was recorded, which invalidates the row rather than the artifact.

References

[1] Obasa AE and Kabanda S. Research ethics committees (recs) perspectives on large language models and ai ethics review: a south african case. *PubMed*, 2026.

[2] F. Bredell, H. A. Engelbrecht, and J. C. Schoeman. Augmenting the action space with conventions to improve multi-agent cooperation in hanabi. *arXiv*, 2024.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv OpenAlex*, 2020.

[4] Anil R. Doshi and Oliver Hauser. Generative ai enhances individual creativity but reduces the collective diversity of novel content. *OpenAlex*, 2024.

[5] Abhiraam Eranti, Yogesh Tewari, Rafael Palacios, and Amar Gupta. Raman spectroscopy pre-trained encoder: A self-supervised learning approach for data-efficient domain-independent spectroscopy analysis. *DOAJ*, 2026.

[6] Nitesh Goyal, Minsuk Chang, and Michael Terry. Designing for human-agent alignment: Understanding what humans want from their agents. *arXiv*, 2024.

[7] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on llm-as-a-judge. *arXiv*, 2024.

[8] Balint Gyevnar, Cheng Wang, Christopher G. Lucas, Shay B. Cohen, and Stefano V. Albrecht. Causal explanations for sequential decision-making in multi-agent systems. *arXiv*, 2023.

[9] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, and Zili Wang. Metagpt: Meta programming for a multi-agent collaborative framework. *Crossref*, 2023.

[10] Sadam Hussain, Mansoor Ali, Farman Ali Pirzado, Masroor Ahmed, and José Gerardo Tamez-Peña. Comparative analysis of deep learning models for breast cancer classification on multimodal data. *Crossref*, 2024.

[11] Sudha Jamthe. Generative ai for enterprise ai. *Crossref*, 2026.

[12] Yixin Ji, Juntao Li, Yang Xiang, Hai Ye, Kaixin Wu, Kai Yao, Jia Xu, Linjian Mo, and Min Zhang. A survey of test-time compute: From intuitive inference to deliberate reasoning. *arXiv Crossref*, 2025.

[13] Eser Kandogan, Sajjadur Rahman, Nikita Bhutani, Dan Zhang, Rafael Li Chen, Kushan Mitra, Sairam Gurajada, Pouya Pezeshkpour, Hayate Iso, Yanlin Feng, Hannah Kim, Chen Shen, Jin Wang, and Estevam Hruschka. A blueprint architecture of compound ai systems for enterprise. *arXiv*, 2024.

[14] Yong-Jae Lee. Optimising distribution-aware genai infrastructure for enterprise knowledge services: supporting seci knowledge flows, digital transformation, and organisational resilience. *Crossref*, 2026.

[15] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, and Weize Chen. Chatdev: Communicative agents for software development. *Crossref*, 2024.

[16] Ju Qin and Yuntao Sun. Fine-tuning clip with dynamic prompt tuning and cross-modal contrastive alignment for multimodal sentiment analysis. *Crossref*, 2026.

[17] Ashish Rana, Michael Oesterle, and Jannik Brinkmann. Gov-rek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems. *arXiv*, 2024.

[18] Arles Rodríguez, Jonatan Gómez, and Ada Diaconescu. A decentralised self-healing approach for network topology maintenance. *arXiv Crossref*, 2020.

[19] Rasmus Ulfsnes, Nils Brede Moe, Viktoria Stray, and Marianne Skarpen. Transforming software development with generative ai: Empirical insights on collaboration and workflow. *arXiv*, 2024.

[20] Fei Wang, Liang Ding, Jun Rao, Ye Liu, Li Shen, and Changxing Ding. Can linguistic knowledge improve multimodal alignment in vision-language pretraining? *arXiv*, 2023.

[21] Ruoxin Xiong, Yael Netser, Pingbo Tang, Beibei Li, and Joonsun Hwang. Socio-technical assessment of generative ai integration in architecture, engineering, and construction (aec) workflows: An empirical study using o*net occupational taxonomy. *Crossref*, 2026.

[22] John Yang, Carlos Jimenez-Gomez, Alexander Wettig, K. Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Crossref*, 2024.

[23] Daohong Yuan. Building an intelligent brain platform for small and medium-sized enterprises using chatglm and multi-agent systems. *PLOS*, 2026.

[24] Changxi Zhu, Mehdi Dastani, and Shihan Wang. A survey of multi-agent deep reinforcement learning with communication. *arXiv*, 2022.